Ders 23 Çalışma Özeti

Ders 23 Çalışma Özeti

Genel Konular

- Dersin tarihi 13 Aralık 2021 Pazartesi. Ders Zoom üzerinden yapıldı.
  - Asıl ders yaklaşık 00:10'da başlar. Öncesi bekleme kısmıdır.
  - Slaytlar Silberschatz, *Operating System Concepts* (9. baskı) 6. bölümüdür.
- Duyuru: 2. vize
  - 21 Aralık 2021 Salı, saat 17:00 (laboratuvar saati). Çoktan seçmeli test olacak.
- Round robin (RR) zamanlama
  - Preemption olan, zaman paylaşımlı (time-sharing) sistemler için tasarlanmış algoritma.
  - Zaman dilimi (time quantum) ve bağlam değişimi (context switch) ilişkisi.
- Çok seviyeli kuyruk (multilevel queue) ve çok seviyeli geri beslemeli kuyruk (multilevel feedback queue)
- İş parçacığı zamanlaması (thread scheduling)
  - Süreç çekişme kapsamı (process contention scope, PCS) ve sistem çekişme kapsamı (system contention scope, SCS).
- Çok işlemcili zamanlama (multiprocessor scheduling)
  - Asimetrik ve simetrik çok işlemcili yapı (SMP), işlemci yakınlığı (processor affinity), NUMA.
  - Yük dengeleme (load balancing): push ve pull migration.
  - Çok çekirdekli işlemciler ve donanım iş parçacıkları (hardware threads).
- Gerçek zamanlı CPU zamanlaması (real-time CPU scheduling)
  - Soft ve hard real-time; kesme gecikmesi (interrupt latency), dağıtım gecikmesi (dispatch latency).
  - Rate monotonic, earliest deadline first (EDF), orantılı pay (proportional share).
- İşletim sistemi örnekleri: Linux ve Windows zamanlayıcıları
- Algoritma değerlendirme: deterministik modelleme, kuyruk modelleri, simülasyon

Hocanın Özellikle Vurguladığı Kısımlar

- 2. vizede baştan sona işlenen her şeyden sorumlusunuz.
  - Uygulama saatlerinde yapılanlar da dahil. Sınav yine çoktan seçmeli test.
- Time quantum seçimi kritiktir.
  - Çok küçük olursa bağlam değişimi sayısı artar, ek yük (overhead) büyür.
  - Çok büyük olursa RR, FCFS'ye döner. Etkileşimli sistemde bekleme artar.
  - Quantum, bağlam değişimi süresine göre büyük olmalıdır. Pratikte 10-100 ms aralığındadır.
- Turnaround time grafiğine bakarken yalnız bir ölçüte baktığınızı unutmayın.
  - Etkileşimli (responsive) bir sistemde asıl ölçüt bekleme süresidir (waiting time).
- Rate monotonic zamanlamada öncelik yalnız periyoda bakar.
  - Toplam CPU kullanımı 1'den küçük olsa bile son tarih (deadline) kaçırılabilir.
- Sistemin hard ya da soft real-time olmasını belirleyen şey, son tarih kaçınca ne olduğudur.
  - Uydu alıcısında kanal geç değişirse ciddi bir şey olmaz: soft real-time.

  - ABS fren sisteminde geç tepki kazaya yol açar: hard real-time.
- Solaris kısmı derste anlatılmadı ama sınavdan sorumlusunuz.

Kısa Tekrar Notları

- RR: her süreç en fazla q kadar çalışır, sonra kuyruğun sonuna gider.
- Örnek: burst süreleri 24, 3, 3 ve $q = 4$ ise ortalama bekleme $17/3 \approx 5{,}67$. FCFS'de aynı sırayla ortalama bekleme 17'dir.
- Kural: CPU burst'lerinin yaklaşık %80'i q'dan kısa olmalı.
- Multilevel queue: süreç girdiği kuyrukta kalır. Kuyruklar arasında sabit öncelik ya da zaman paylaşımı (ör. %80 / %20) olur.
- Multilevel feedback queue: süreç kuyruklar arasında yer değiştirebilir. Uzun CPU kullanan aşağıya iner.
- PCS: thread kütüphanesi, kullanıcı thread'lerini LWP'ler üzerinde zamanlar. SCS: çekirdek, tüm çekirdek thread'leri arasında zamanlar.
- Linux ve macOS yalnız `PTHREAD_SCOPE_SYSTEM` destekler.
- Soft affinity: aynı işlemcide tutmaya çalışır, garanti vermez. Hard affinity: çalışabileceği işlemcileri sınırlar.
- Push migration: ayrı bir görev yükü düzenli kontrol edip süreç taşır. Pull migration: boştaki işlemci, dolu kuyruktan iş çeker.
- Gerçek zamanlı görev: periyot p, son tarih d, çalışma süresi t; $0 \leq t \leq d \leq p$.
- Rate monotonic: periyot ne kadar kısa ise öncelik o kadar yüksek. En kötü durum kullanım sınırı $N(2^{1/N} - 1)$.
- EDF: son tarihi en yakın olan en yüksek önceliği alır. Teorik olarak optimaldir.
- Linux CFS: en küçük sanal çalışma süresine (vruntime) sahip görevi seçer. Nice değeri -20 ile +19 arasındadır.
- Windows: 32 öncelik seviyesi. 1-15 değişken sınıf, 16-31 gerçek zamanlı sınıf.

Detaylı Açıklamalar

Round robin ve time quantum

RR, FCFS'ye preemption eklenmiş hâli gibi düşünülebilir. Hazır kuyruğu dairesel bir kuyruktur. Her süreç en fazla bir time quantum kadar CPU kullanır. Burst'ü quantum'dan kısa olan süreç, quantum dolmadan bırakır ve sıradaki süreç gelir.

Slayttaki örnek: P1 = 24, P2 = 3, P3 = 3 ve $q = 4$.

Süreç	Bitiş	Bekleme
P1	30	6
P2	7	4
P3	10	7

Ortalama bekleme $17/3 \approx 5{,}67$ birimdir. Aynı süreçler FCFS ile çalışınca bekleme süreleri 0, 24 ve 27 olur; ortalama 17'dir.

Slayttaki grafik, burst'leri 6, 3, 1, 7 olan dört süreç için ortalama turnaround süresini gösterir. Değerler Python ile hesaplandı:

q	1	2	3	4	5	6	7
Ort. turnaround	11,0	11,5	10,75	11,5	12,25	10,5	10,5

Turnaround, quantum büyüdükçe düzenli olarak iyileşmez. Hoca, quantum en uzun burst'e eşit olunca turnaround'un düştüğünü gösterdi. Ama burada bekleme süresine bakılmadığını vurguladı.

Çok seviyeli kuyruklar

Multilevel queue'da süreçler gruplara ayrılır. Örneğin ön plan (foreground, etkileşimli) ve arka plan (background, batch). Her kuyruğun kendi algoritması olabilir: ön planda RR, arka planda FCFS. Kuyruklar arasında da bir zamanlama gerekir:

- Sabit öncelik: arka plan, ancak ön plan kuyruğu boşsa çalışır. Açlık (starvation) riski vardır.
- Zaman dilimi paylaşımı: ön plan CPU zamanının %80'ini, arka plan %20'sini alır.

Multilevel feedback queue'da süreç kuyruklar arasında hareket edebilir. Slayttaki örnekte üç kuyruk vardır:

- Q_0: RR, $q = 8$.
- Q_1: RR, $q = 16$.
- Q_2: FCFS.

Yeni süreç Q_0'a girer. 8 birimde bitmezse Q_1'in sonuna iner. Orada da 16 birimde bitmezse Q_2'ye iner. Q_1 ancak Q_0 boşsa, Q_2 ancak Q_0 ve Q_1 boşsa çalışır. Böylece kısa işler öne geçer, SJF'ye benzer bir davranış elde edilir.

Bu yapıyı tanımlayan parametreler: kuyruk sayısı, her kuyruğun algoritması, süreci yükseltme ve indirme kuralları, yeni sürecin hangi kuyruğa gireceği.

İş parçacığı zamanlaması

Many-to-one ve many-to-many modellerinde thread kütüphanesi, kullanıcı thread'lerini hafif süreçler (lightweight process, LWP) üzerinde zamanlar. Bu rekabet aynı sürecin thread'leri arasındadır: PCS. Bir thread'in LWP'yi alması, CPU'da çalıştığı anlamına gelmez. Çekirdek thread'inin de CPU için tüm çekirdek thread'leri arasında yarışması gerekir: SCS. One-to-one model kullanan Linux ve Windows yalnız SCS kullanır.

POSIX'te `pthread_attr_setscope` ve `pthread_attr_getscope` ile kapsam belirlenebilir. Linux ve macOS'ta yalnız `PTHREAD_SCOPE_SYSTEM` çalışır.

Çok işlemcili zamanlama

- Asimetrik çok işlemcilik: zamanlama kararlarını tek bir işlemci verir. Paylaşılan çekirdek veri yapılarına tek işlemci eriştiği için senkronizasyon derdi azdır.
- Simetrik çok işlemcilik (SMP): her işlemci kendi zamanlamasını yapar. Ortak hazır kuyruğu varsa senkronizasyon gerekir. Her işlemciye ayrı kuyruk verilirse yük dengesizliği oluşabilir.

İşlemci yakınlığı: süreç bir işlemcide çalışırken önbellek (cache) o sürecin verisiyle dolar. Süreç başka işlemciye taşınırsa eski önbellek geçersiz olur, yeni önbellek baştan dolar. Bu yüzden sistemler süreci aynı işlemcide tutmaya çalışır.

NUMA (non-uniform memory access) mimarisinde her işlemcinin hızlı eriştiği bir bellek bölgesi vardır. Diğer bölgelere erişim yavaştır. Bu durumda affinity daha da önemli olur.

Yük dengeleme her iki yöntemle de yapılır ve ek yükü vardır. Süreç taşımak önbellek etkisini bozduğu için kazanç her zaman maliyetten büyük olmayabilir.

Çok çekirdekli işlemcilerde bellek erişimi sırasında işlemci bekler (memory stall). Bir çekirdeğe iki donanım thread'i konursa birinin bellek beklemesi, diğerinin hesaplamasıyla örtüşür. İşletim sistemi her donanım thread'ini ayrı bir mantıksal işlemci olarak görür.

Gerçek zamanlı zamanlama

Gerçek zamanlı sistem, görevlerin belirli süreler içinde tamamlanması gereken, zamana duyarlı sistemdir. Zamanlama önceden (offline) ya da çalışırken (online) yapılabilir.

Gecikmeyi etkileyen iki süre vardır:

- Kesme gecikmesi: kesmenin oluşmasından, kesme servis rutininin başlamasına kadar geçen süre.

- Dağıtım gecikmesi: çalışan süreci CPU'dan alıp diğerini koyma süresi. İçinde çakışma (conflict) aşaması vardır: düşük öncelikli sürecin tuttuğu kaynakların serbest bırakılması gerekir.

Rate monotonic: Öncelik periyotla ters orantılıdır ve statiktir. Yüksek öncelikli görev gelince düşük öncelikli görev kesilir.

- Örnek 1: P1 ($p = 50$, $t = 20$), P2 ($p = 100$, $t = 35$). Kullanım $20/50 + 35/100 = 0{,}75$. Görevler son tarihlerini kaçırmaz.
- Örnek 2: P1 ($p = 50$, $t = 25$), P2 ($p = 80$, $t = 35$). Kullanım $25/50 + 35/80 = 0{,}9375$. Yine de P2 ilk son tarihini (80) kaçırır, 85'te biter. Python simülasyonuyla doğrulandı.
- İki görev için en kötü durum sınırı $2(\sqrt{2} - 1) \approx 0{,}83$. N büyüdükçe sınır $\ln 2 \approx 0{,}69$'a iner.

EDF: Son tarihi en yakın olan görev en yüksek önceliği alır; öncelik dinamiktir. Örnek 2'deki görevler EDF ile hiç son tarih kaçırmadan çalışır. EDF teorik olarak optimaldir: EDF'nin zamanlayamadığı bir görev kümesini başka bir algoritma da zamanlayamaz.

Orantılı pay: CPU zamanı T paya bölünür. Örnekte A'ya 50, B'ye 15, C'ye 20 pay verilir, toplam 85'tir. Kalan 15 pay, sonradan gelecek süreçler için boş bırakılır. Ağırlıklı round robin'e benzer.

POSIX gerçek zamanlı zamanlama için `SCHED_FIFO` ve `SCHED_RR` sunar. Hoca ayrıntılarını atladı.

Linux ve Windows

- Linux 2.5 öncesinde klasik UNIX zamanlayıcısı kullanılıyordu.
- 2.5 ile $O(1)$ zamanlayıcıya geçildi: öncelik tabanlı, preemptive. Gerçek zamanlı aralık 0-99, nice aralığı 100-139. Etkileşimli süreçlerde tepki süresi zayıf kaldı.
- 2.6.23 ile Completely Fair Scheduler (CFS) geldi:
  - Zamanlama sınıfları vardır; varsayılan sınıf ve gerçek zamanlı sınıf.
  - Her görevin sanal çalışma süresi (vruntime) tutulur. Görevler vruntime'a göre kırmızı-siyah ağaçta durur. En küçük vruntime'lı görev seçilir.
  - Nice değeri -20 ile +19 arasındadır. Nice ne kadar küçükse öncelik o kadar yüksektir. Normal öncelik 120, nice -20 için 100, nice +19 için 139'dur.
- Windows zamanlayıcısının adı dispatcher'dır. 32 seviyeli öncelik vardır: 1-15 değişken sınıf, 16-31 gerçek zamanlı sınıf. Thread; kendini bloklayana, zaman dilimi bitene ya da daha yüksek öncelikli bir thread gelene kadar çalışır.

Algoritma değerlendirme

- Deterministik modelleme: belirli bir iş yükü için Gantt şemasıyla her algoritmanın ölçütünü hesaplamak. Hızlıdır ama iş yükünü tam bilmek gerekir.
- Kuyruk modelleri: sistemi kuyruk ağı olarak modelleyip hesaplamak.
- Simülasyon: gerçek sistemden alınan izlerle (trace) farklı algoritmaları denemek. Daha gerçekçi sonuç verir.

Ders sonrası notlar

- Altyazıdaki "complete fair queuing" ifadesi Linux'un CPU zamanlayıcısını kastediyor. Zamanlayıcının adı Completely Fair Scheduler'dır (CFS). CFQ (Completely Fair Queuing) Linux'ta bir disk G/Ç zamanlayıcısıdır.
- Hoca $O(1)$ zamanlayıcının üstünlüğünü "lineer zamanda çalışması" diye anlattı. Doğrusu: Seçim kararını süreç sayısından bağımsız, sabit zamanda ($O(1)$) verir. Önceki zamanlayıcı ise lineer zamanlıydı.
- EDF anlatılırken önce "deadline ne kadar uzaksa önceliği o kadar yüksek" dendi, hemen ardından düzeltildi. Doğrusu: son tarihi ne kadar yakınsa öncelik o kadar yüksektir.
- RR örneğinde $q = 4$ için 7 bağlam değişimi sayıldı. Python ile sayıldığında bu sayı Gantt şemasındaki bütün dilim sınırlarını sayıyor. P1'in yine P1'e geçtiği sınırlar sayılmazsa, farklı süreçler arası geçiş 3'tür. $q = 1$'de dilim sınırı 29'a, farklı süreçler arası geçiş 9'a çıkar. Hocanın "20 küsür" dediği sayı dilim sınırlarıdır.

- Hoca kuralı “süreçlerin %80’inin burst’ünden uzun bir quantum” diye anlattı. Slayttaki kural: CPU burst’lerinin %80’i q’dan kısa olmalı. Aynı anlama gelir. Örnekte $q = 6$ seçilince dört burst’ün üçü (%75) quantum’a sığar; hoca bu oranı “%83” diye söyledi gibi duyuluyor.
- Turnaround grafiğindeki değerler: $q = 3$ için 10,75 ve $q = 5$ için 12,25. Derste “11’in altı” ve “12” dendi.
- Rate monotonic Örnek 2’de hoca “35 bölü 80, 0,94” dedi. $35/80 = 0{,}4375$’tir; 0,94 (tam olarak 0,9375) iki görevin toplam kullanımıdır.
- Rate monotonic bound’u derste formül olarak yazılmadı. Doğru formül $N(2^{1/N} - 1)$’dir.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.